Universidad Nacional de San Martín

TP Integrador Base de Datos

Materia	Base de Datos
Carrera	Tecnicatura en Programación Informática
Docentes	Gabriel FLORES - Rodrigo CASCO
Proyecto	Alero: sistema de reservas y reseñas para alojamientos
Integrantes	Milena Maddaleni, Tadeo Valentin Fernandez, Cristhiam Mauricio Siachoque y Juan Ignacio Estevez
Fecha de entrega	13 de noviembre de 2026
Repositorio	https://github.com/ju4nchp/tp-integrador-bd-alero
Aplicación publicada	https://alero-tp-bd.onrender.com

1. Descripción del problema

Elegimos el dominio de las reservas de viajes. Un alojamiento recibe reservas, algunas se cancelan y, cuando la estadía termina, el huésped puede dejar una reseña. El sistema lo usa el dueño o encargado del alojamiento para cargar, consultar, modificar, confirmar y cancelar reservas, registrar las reseñas que dejan los huéspedes y responderlas, y ver estadísticas de facturación, cancelaciones y puntajes.

Definimos estas reglas: una reserva nueva empieza pendiente y se confirma desde su detalle; solo se modifican o cancelan las pendientes o confirmadas; las reservas no se borran (la baja es lógica y quedan en un historial); y las reseñas son solo de estadías completadas, una por reserva, y no se editan ni se borran. Cancelar y dar de baja no son lo mismo: se cancela cuando el huésped no viene (se elige un motivo y cuenta en la tasa de cancelación) y se da de baja una reserva que se cargó por error (deja de contar en las estadísticas).

2. Descripción y origen del dataset

Usamos el archivo **Travel_And_Tourism.csv**. Origen: (completar con el link de donde lo descargamos). Tiene 1000 filas y 29 columnas: cada fila es una reserva y trae todo junto (cliente, destino, hotel, viaje, pago, estado y, si corresponde, la reseña o el motivo de cancelación). Hay

1000 clientes, 78 hoteles, 26 ciudades de destino en 10 países y 44 ciudades en total. De las reservas, 772 están completadas, 141 canceladas, 79 confirmadas y 8 pendientes.

Grupo	Columnas del CSV
Reserva	Booking_ID, Booking_Date, Travel_Date, Return_Date, Booking_Status
Cliente	Customer_ID, Customer_Name, Customer_Age, Gender, Customer_City, Customer_Country
Destino y hotel	Destination_City, Destination_Country, Hotel_Name, Hotel_Rating
Viaje	Number_of_Travellers, Number_of_Adults, Number_of_Children, Number_of_Rooms, Number_of_Nights, Meal_Plan, Transportation_Type
Pago	Payment_Method, Discount_Amount, Total_Trip_Cost
Cancelación y reseña	Cancellation_Status, Cancellation_Reason, Customer_Rating, Customer_Review_Comment

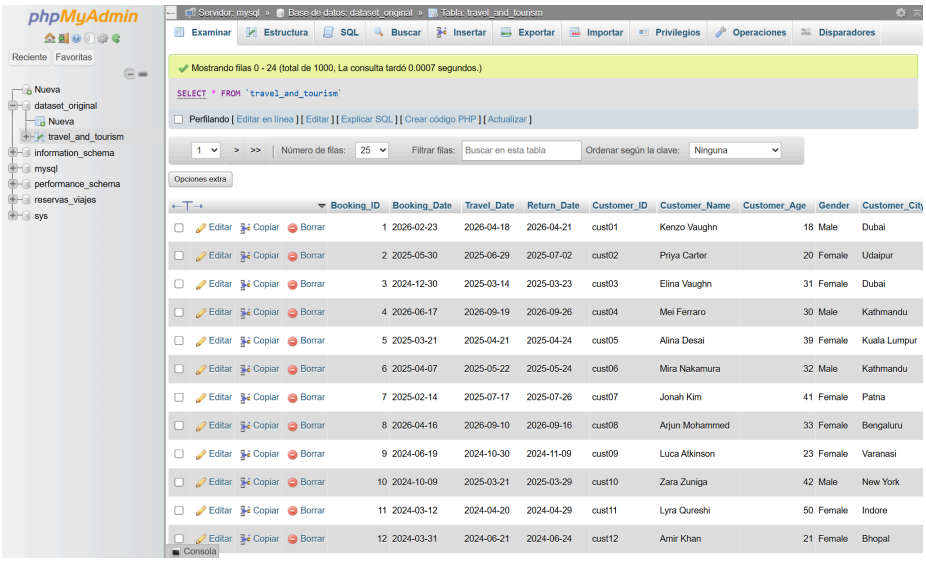


Figura 1. El CSV cargado tal cual en MySQL: una sola tabla de 29 columnas.

3. Análisis inicial de los datos

Cargamos el CSV tal cual en una tabla de MySQL (base dataset_original) y lo revisamos con consultas en phpMyAdmin (script database/analisis_dataset.sql). Encontramos:

- **Datos repetidos:** el nombre, la ciudad y el país del hotel se repiten en cada reserva (Cedar Heights aparece en 25 filas). Lo mismo pasa con la ciudad y el país del cliente, y con textos como el régimen, el medio de pago o el motivo de cancelación.
- **Dependencias:** Customer_ID determina nombre, edad, género y ciudad; el hotel determina su ciudad, y la ciudad determina el país. Las tomamos como reglas del dominio y no solo como algo que muestran los datos: un hotel está en una sola ciudad y una ciudad pertenece a un solo país (en el CSV ninguna ciudad tiene más de un país, Figura 2). Las dos últimas son dependencias transitivas.
- **Columnas que se calculan:** viajeros = adultos + niños, noches = regreso − salida, y el estado de cancelación sale del estado de la reserva. Coinciden en las 1000 filas (Figura 3).

- **Valores vacíos disfrazados:** 859 motivos de cancelación dicen "Not Applicable" y 228 puntajes valen 0 porque no hay reseña. La cancelación y la reseña son datos opcionales.
- **Hotel_Rating** cambia en cada reserva del mismo hotel (de 1.0 a 5.0): es el puntaje que pone el cliente, no un dato fijo del hotel.

Con todo en una tabla hay problemas de **actualización** (cambiar el nombre de un hotel obliga a tocar todas sus filas), de **inserción** (no se puede cargar un hotel sin una reserva) y de **eliminación** (si se borra la única reserva de un cliente se pierden sus datos; en el CSV cada cliente tiene una sola reserva).

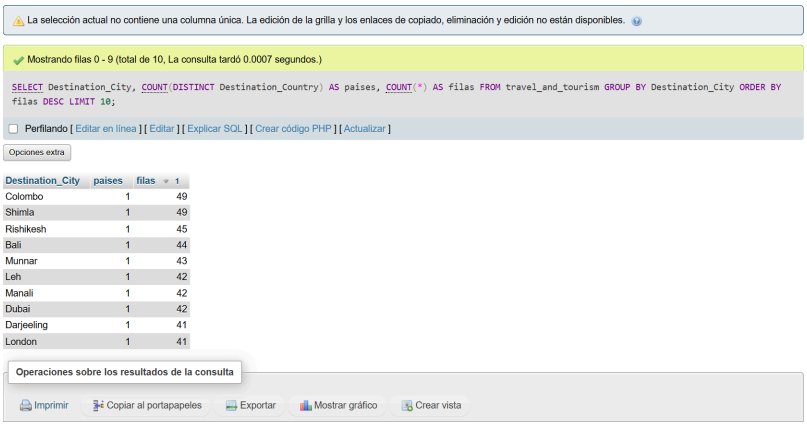


Figura 2. Cada ciudad tiene un solo país (columna paises en 1).

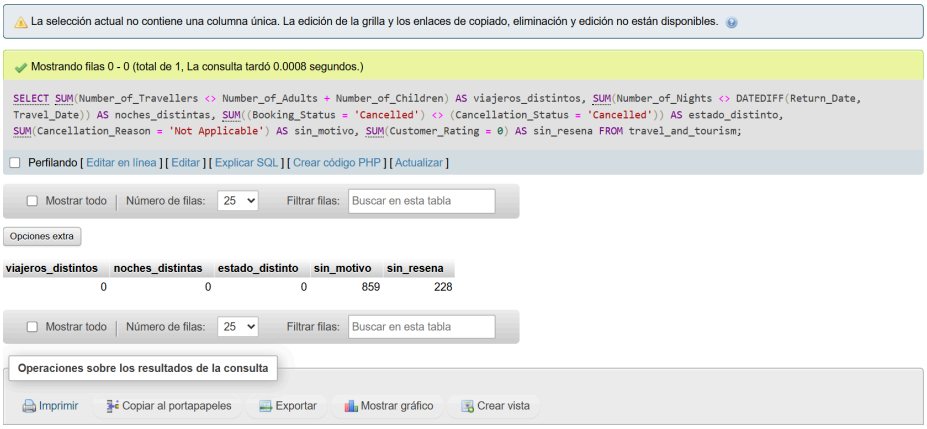


Figura 3. Las columnas calculadas no tienen diferencias, y hay 859 "Not Applicable" y 228 puntajes en 0.

4. Proceso de normalización

Paso	Cómo quedó
Origen	1 tabla de 29 columnas, una fila por reserva.
1FN	Ya cumple: valores atómicos, sin grupos repetidos y Booking_ID único.
2FN	Ya cumple: la clave es una sola columna, así que no hay dependencias parciales.
3FN	No cumple por las dependencias transitivas: separamos país, ciudad, cliente y hotel, y los textos que se repiten pasan a catálogos.
Resultado	11 tablas en MySQL con PK, FK y restricciones CHECK. Las reseñas van a MongoDB (punto 7).

Las dependencias transitivas que eliminamos son:

```
Booking_ID → Customer_ID → Customer_Name, Customer_Age, Gender, Customer_City → Customer_Country
Booking_ID → Hotel_Name → Destination_City → Destination_Country
```

- **PAIS** (nombre, region), **CIUDAD** (id, nombre, pais_nombre), **CLIENTE** y **HOTEL** quedan en tablas propias. Una sola tabla CIUDAD sirve para clientes y hoteles. La región no venía en el CSV y la agregamos para agrupar estadísticas.
- Meal_Plan, Payment_Method, Transportation_Type, Booking_Status y Cancellation_Reason pasan a catálogos (regimen, medio_pago, transporte, estado_reserva y motivo_cancelacion) para no repetir texto.
- No guardamos lo que se calcula: viajeros, noches, estado de cancelación y el total a pagar (total_trip_cost − discount_amount).
- **CANCELACION** existe solo para las 141 reservas canceladas, así no se guardan los "Not Applicable". Los puntajes y el comentario de la reseña tampoco quedan en MySQL: van a MongoDB. Por eso el hotel no tiene una columna de puntaje: el promedio se calcula con las reseñas cada vez que se consulta, así incluye también las reseñas nuevas.

5. Diagrama Entidad-Relación

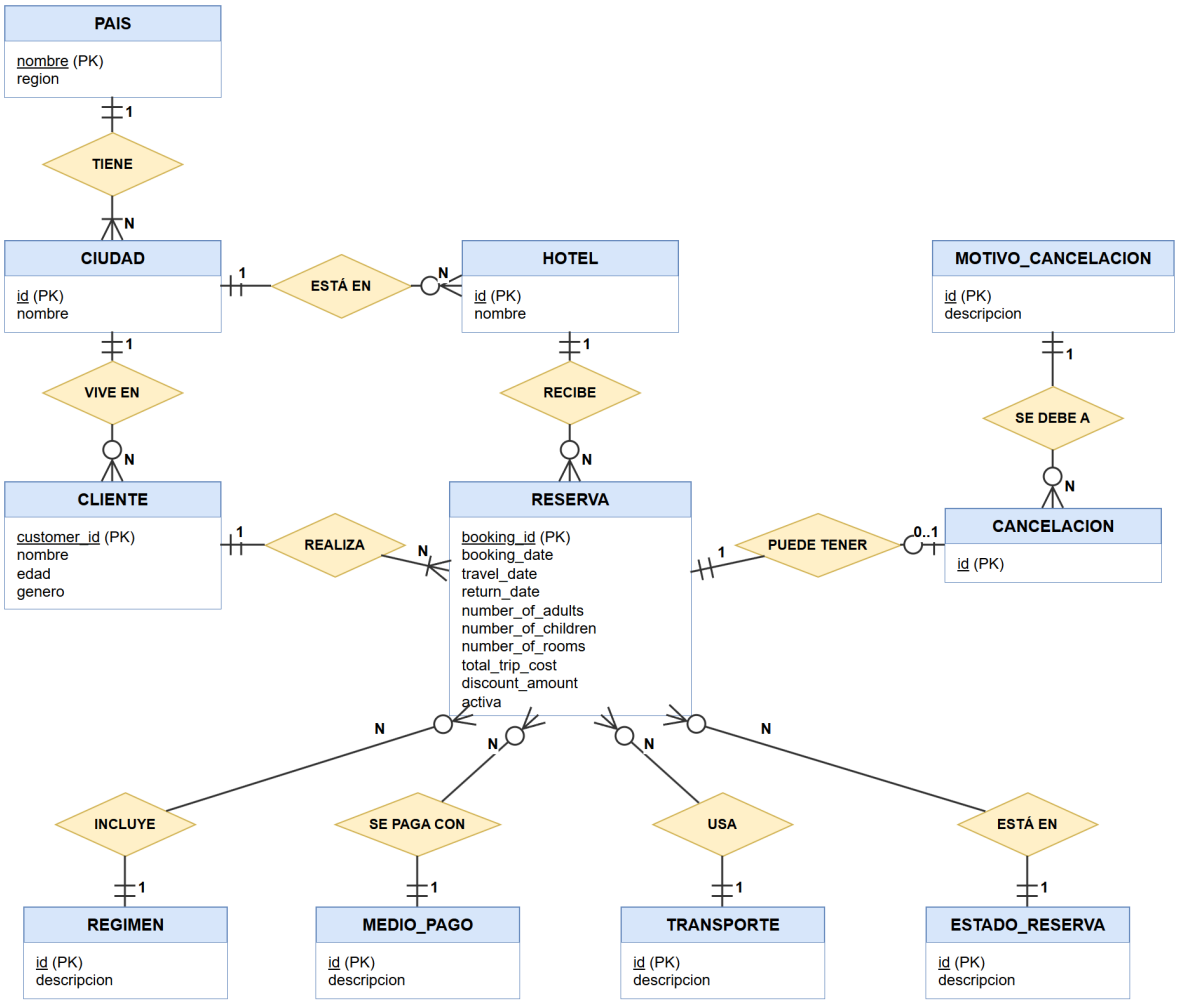


Figura 4. DER de la base reservas_viajes, hecho en draw.io (documentacion/DER.drawio).

Lo dibujamos en draw.io siguiendo la guía de la cátedra: las entidades van en rectángulos con sus atributos (la clave subrayada y marcada PK), las relaciones en rombos con un verbo, y la cardinalidad con patas de gallo y los números 1 y N. En el DER no van las claves foráneas: aparecen recién en el modelo relacional (punto 6). Un país tiene una o más ciudades, y en cada ciudad viven clientes y están los hoteles (1:N). Un cliente realiza reservas y un hotel las recibe: RESERVA resuelve la relación N:M entre CLIENTE y HOTEL. Cada reserva incluye un régimen, se paga con un medio de pago, usa un transporte y está en un estado (N:1). Una reserva puede tener una cancelación o ninguna (1:0..1), y cada cancelación se debe a un motivo del catálogo MOTIVO_CANCELACION (N:1). Las reseñas no están en el DER porque se guardan en MongoDB: cada reseña apunta a su reserva con el número de reserva (reservaId).

6. Modelo relacional

database/modelo_relacional.sql crea la base reservas_viajes con las 11 tablas. Todas usan InnoDB, el motor de almacenamiento de MySQL que soporta claves foráneas y transacciones. Para pasar

del DER al modelo relacional seguimos las reglas de mapeo: cada entidad es una tabla, la PK del lado 1 pasa como FK a la tabla del lado N y la relación N:M entre cliente y hotel queda en la tabla reserva (Figura 5).

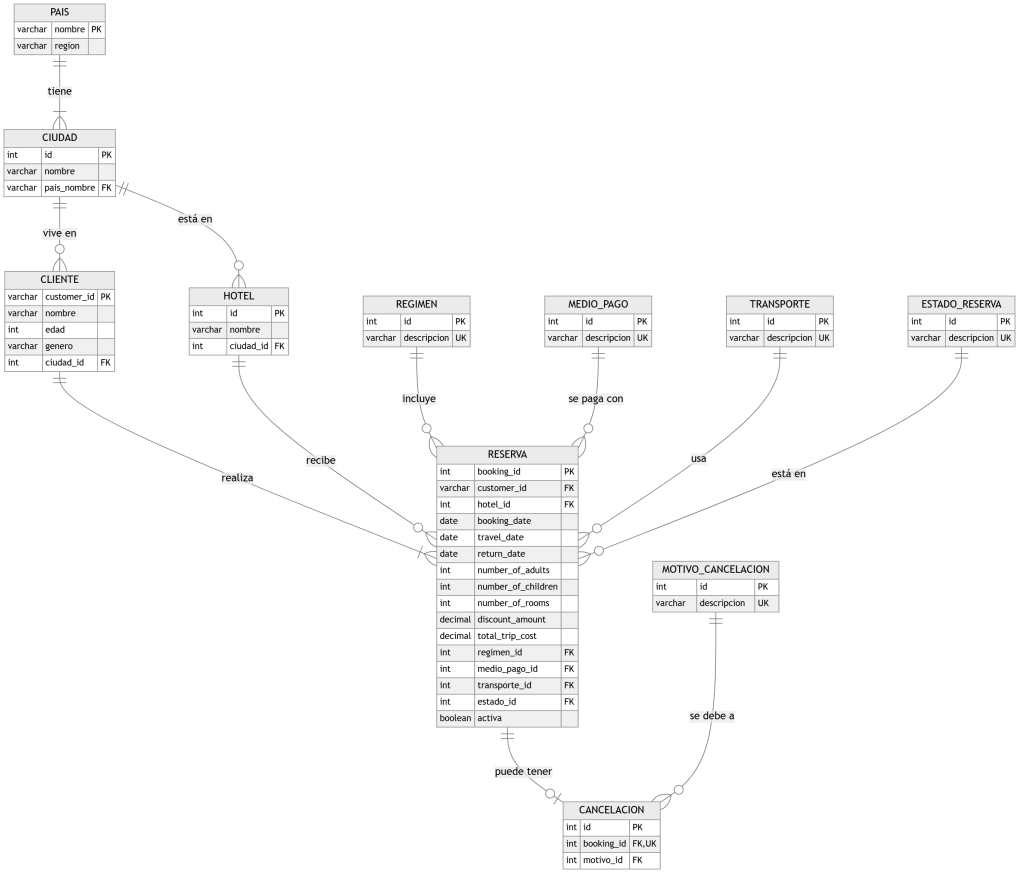


Figura 5. El modelo relacional: cada tabla con sus columnas, la PK, las FK y las relaciones.

Tabla	Columnas	PK / FK	Restricciones
pais	nombre VARCHAR(100), region VARCHAR(100)	PK nombre	region NOT NULL
ciudad	id INT, nombre VARCHAR(100), pais_nombre VARCHAR(100)	PK id; FK pais_nombre → pais	UNIQUE (nombre, pais_nombre)
cliente	customer_id VARCHAR(50), nombre VARCHAR(150), edad INT, genero VARCHAR(20), ciudad_id INT	PK customer_id; FK ciudad_id → ciudad	edad ≥ 18; genero 'Male' o 'Female'
hotel	id INT, nombre VARCHAR(150), ciudad_id INT	PK id; FK ciudad_id → ciudad	UNIQUE (nombre, ciudad_id)
regimen, medio_pago, transporte, estado_reserva, motivo_cancelacion	id INT, descripcion VARCHAR(100)	PK id	descripcion NOT NULL y UNIQUE
reserva	booking_id INT, customer_id, hotel_id, booking_date, travel_date, return_date DATE, number_of_adults, number_of_children, number_of_rooms INT, discount_amount, total_trip_cost DECIMAL(10,2), regimen_id, medio_pago_id, transporte_id, estado_id INT, activa BOOLEAN	PK booking_id; FK a cliente, hotel y los 4 catálogos	reserva ≤ salida < regreso; adultos ≥ 1; niños ≥ 0; habitaciones entre 1 y los viajeros; costo > 0; descuento entre 0 y el costo
cancelacion	id INT, booking_id INT, motivo_id INT	PK id; FK booking_id → reserva; FK motivo_id → motivo_cancelacion	booking_id UNIQUE (una cancelación por reserva)

Los id de ciudad, hotel, cancelación y los catálogos son AUTO_INCREMENT. Las FK usan RESTRICT: no se puede borrar una fila que otra tabla usa.

Carga de los datos. database/datos_iniciales.sql hace todo en SQL. Primero copia el CSV a la tabla plana dataset_original.travel_and_tourism con LOAD DATA INFILE. Después llena cada tabla con INSERT ... SELECT: DISTINCT para quedarse con un valor por entidad y JOIN para buscar los id. Por ejemplo, la tabla hotel:

```
INSERT INTO hotel (nombre, ciudad_id)
SELECT DISTINCT t.Hotel_Name, c.id
  FROM dataset_original.travel_and_tourism t
  JOIN ciudad c ON c.nombre = t.Destination_City AND c.pais_nombre = t.Destination_Country
 ORDER BY t.Hotel_Name;
```

La región sale de un CASE según el país, y la tabla cancelacion se llena solo con las reservas canceladas, buscando el id del motivo en el catálogo (Figuras 6 a 8).

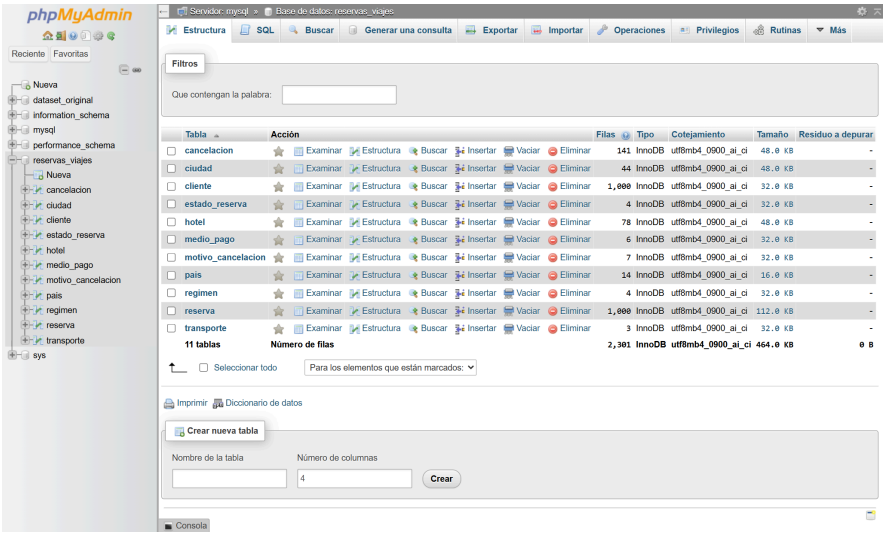


Figura 6. Las 11 tablas en phpMyAdmin con los datos cargados (todas InnoDB).

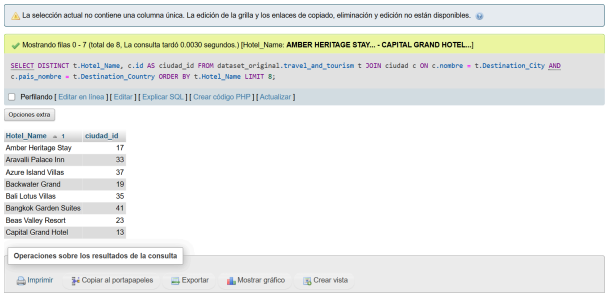


Figura 7. El SELECT de la carga de hotel: cada hotel una sola vez, con el id de su ciudad.

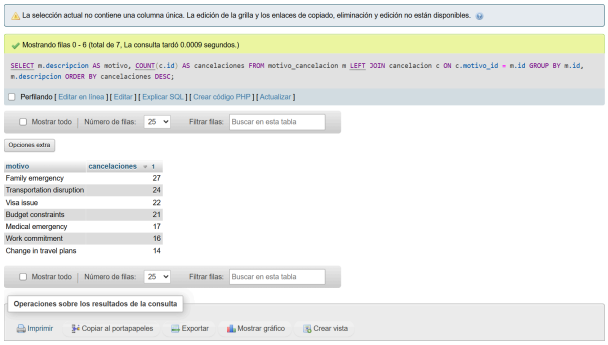


Figura 8. El catálogo motivo_cancelacion y cuántas cancelaciones tiene cada motivo.

Probamos que las restricciones funcionan: MySQL rechaza una reserva que vuelve antes de salir (Figura 9) y no deja borrar una reserva que tiene una cancelación (Figura 10).

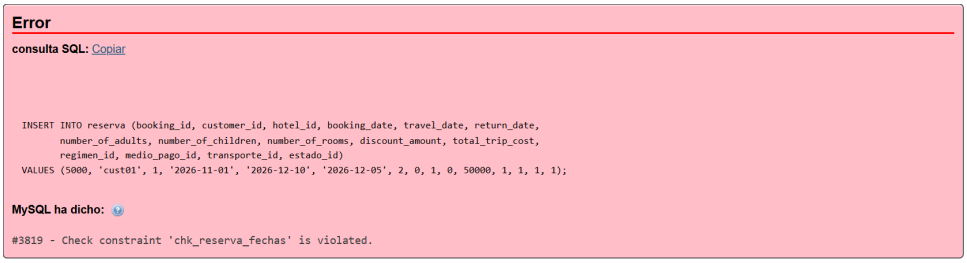


Figura 9. La restricción chk_reserva_fechas rechaza la reserva.

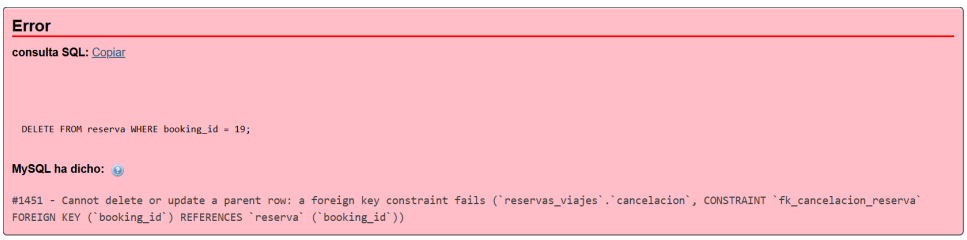


Figura 10. La clave foránea de cancelacion impide borrar la reserva 19.

7. Diseño de la solución NoSQL

Usamos **MongoDB**, una base documental, solo para las reseñas. La colección **resenas** tiene un documento por reseña: 772, una por cada estadía completada del CSV. Cada documento guarda reservaId (el número de la reserva en MySQL), hotelId, el nombre y el destino del hotel, los dos puntajes, el comentario, un array de etiquetas y un objeto respuestaAgencia, que aparece solo cuando el alojamiento contesta (Figura 11).

El archivo resenas.json lo generamos con una consulta SQL (database/scripts_nosql/resenas.sql, con JSON_OBJECT) sobre la tabla del CSV y las tablas ya cargadas, así el hotelId coincide con el id del hotel en MySQL. Después se importa con mongoimport.

Como MongoDB no tiene FK ni CHECK, el script indices.js agrega un validador $jsonSchema (campos obligatorios, tipos, comentario de hasta 255 caracteres y puntajes entre 1 y 5; Figura 12) y dos índices: uno único en reservaId, para que haya una sola reseña por reserva, y otro en hotelId, para buscar y agrupar por alojamiento. Lo que MongoDB no puede controlar, que la reserva exista en MySQL y esté completada, lo controla el backend antes de guardar.

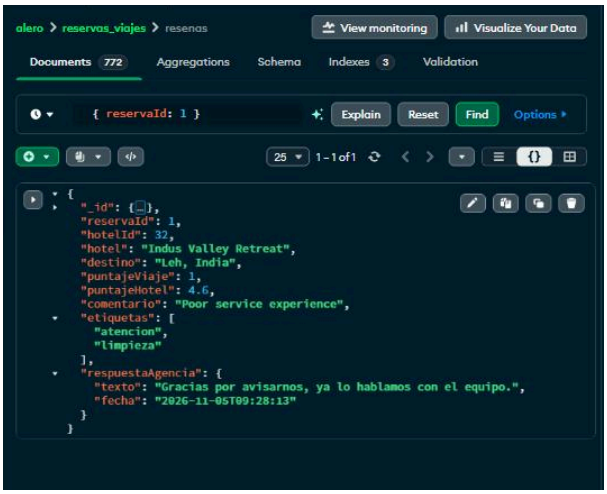


Figura 11. Una reseña en MongoDB Atlas, con etiquetas y respuesta.

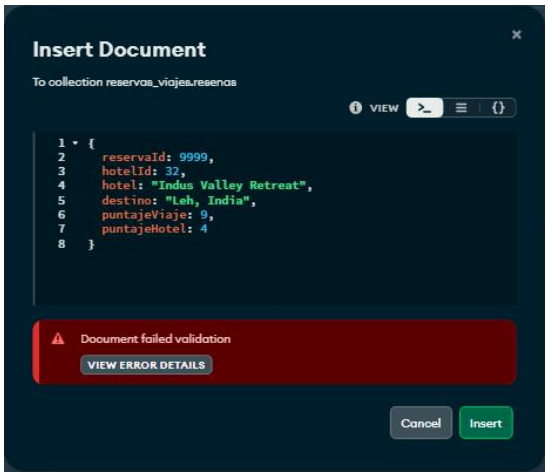


Figura 12. El validador rechaza un puntaje de 9.

8. Justificación de la utilización de NoSQL

Las **reseñas** son la parte del sistema que más se beneficia de NoSQL. Las etiquetas son una lista de largo variable: en MongoDB es un array, y en MySQL sería otra tabla y un JOIN. La respuesta del alojamiento es opcional: en MongoDB el objeto aparece solo si existe, y en MySQL serían dos columnas casi siempre en NULL. Además, una reseña se escribe una vez y no se edita (solo se le agrega la respuesta o las etiquetas), y se lee siempre completa, como una ficha. Por eso elegimos una base documental.

- **Ventajas:** esquema flexible para listas y datos opcionales, cada reseña se lee en un solo acceso y los promedios por alojamiento salen de un aggregate sobre la colección, sin tocar las tablas de reservas.
- **Desventajas:** el nombre y el destino del hotel quedan copiados en cada reseña (si un hotel cambia de nombre hay que actualizar varias reseñas, justo la anomalía que la 3FN evita), la

integridad depende del validador y del backend, y no hay JOIN ni transacciones entre las dos bases.

Cómo conviven los dos motores: cada dato está en una sola base. MySQL guarda lo que necesita integridad fuerte (clientes, hoteles, reservas, cancelaciones y catálogos) y MongoDB guarda las reseñas. Las dos funcionan al mismo tiempo detrás del mismo backend, que es el que las relaciona.

9. Arquitectura general de la aplicación

```
FRONTEND (React)
    |
BACKEND (Kotlin + Spring Boot): controladores y servicios con las reglas
    |
Capa de persistencia: ReservaRepository, CatalogoRepository, ResenaRepository
    /                                   \
MySQL: reservas_viajes (11 tablas)     MongoDB: colección resenas
```

El frontend nunca accede a las bases: todo pasa por la API del backend. En el menú de la aplicación se ve qué base usa cada sección (reservas: MySQL, reseñas: MongoDB, estadísticas: las dos). Todo se levanta con Docker Compose, y hay una versión en la nube con planes gratuitos: Render para la aplicación, Aiven para MySQL y MongoDB Atlas.

10. Explicación del backend polimórfico

Los servicios, que tienen las reglas de negocio, no saben en qué base está cada dato: usan solo las interfaces de la capa de persistencia. Cada interfaz tiene una implementación para un motor: MySqlReservaRepository y MySqlCatalogoRepository usan Spring Data JPA y consultas SQL, y MongoResenaRepository usa MongoTemplate. Qué implementación se usa lo define la configuración de Spring, así que pasar una parte del sistema a otro motor es escribir otra implementación de la interfaz, sin tocar los servicios ni el frontend.

En la primera versión cada interfaz tenía dos implementaciones, los datos estaban completos en las dos bases y el frontend elegía el motor con un selector. Lo cambiamos porque la idea del trabajo es que los dos motores convivan, cada uno con la parte de los datos que le conviene (un backend políglota). Ahora el backend trabaja con las dos bases a la vez, y hay dos operaciones que usan las dos:

- **Registrar una reseña:** el servicio busca la reserva en MySQL y controla que exista, esté activa y completada; después se fija en MongoDB que no tenga reseña, y recién ahí la guarda.
- **Estadísticas por alojamiento:** las reservas, cancelaciones y facturación salen de MySQL con un GROUP BY por hotel, y los promedios de las reseñas salen de MongoDB con un $group por hotelId. El backend junta los dos resultados por el id del hotel.

Sobre las reservas (MySQL) hay alta, consulta, listado, modificación, confirmación, cancelación y baja lógica. Sobre las reseñas (MongoDB) hay alta, consulta, listado, respuesta y etiquetas.

11. Tecnologías utilizadas

MySQL 8.4 (motor InnoDB) con phpMyAdmin, MongoDB 7 (MongoDB Atlas en la nube), SQL para la carga (LOAD DATA INFILE e INSERT ... SELECT) y mongoimport para las reseñas, Kotlin con Spring Boot (Spring Data JPA y Spring Data MongoDB) para el backend, React con Vite y TypeScript para el frontend, Docker Compose, Git y GitHub.

12. Instrucciones para ejecutar el proyecto

Con Docker abierto, en la carpeta del proyecto: `docker compose up --build`. La primera vez, MySQL corre modelo_relacional.sql y datos_iniciales.sql, y MongoDB importa resenas.json y corre indices.js. La aplicación queda en http://localhost:5173 y phpMyAdmin en http://localhost:8081. También se puede usar sin instalar nada en https://alero-tp-bd.onrender.com: la primera carga tarda porque el plan gratuito apaga el servidor cuando no se usa. El README.md tiene el detalle.

13. Ejemplos de funcionamiento

Consulta en MySQL: facturación y tasa de cancelación por país de destino. Usa cuatro tablas con JOIN, un LEFT JOIN para las cancelaciones y la facturación ya con el descuento:

```
SELECT p.nombre AS pais, p.region, COUNT(*) AS reservas, COUNT(c.booking_id) AS canceladas,
       SUM(CASE WHEN c.booking_id IS NULL THEN r.total_trip_cost - r.discount_amount ELSE 0 END)
AS facturacion
  FROM reserva r
  JOIN hotel h ON h.id = r.hotel_id
  JOIN ciudad ci ON ci.id = h.ciudad_id
  JOIN pais p ON p.nombre = ci.pais_nombre
  LEFT JOIN cancelacion c ON c.booking_id = r.booking_id
 WHERE r.activa = TRUE
 GROUP BY p.nombre, p.region ORDER BY facturacion DESC;
```

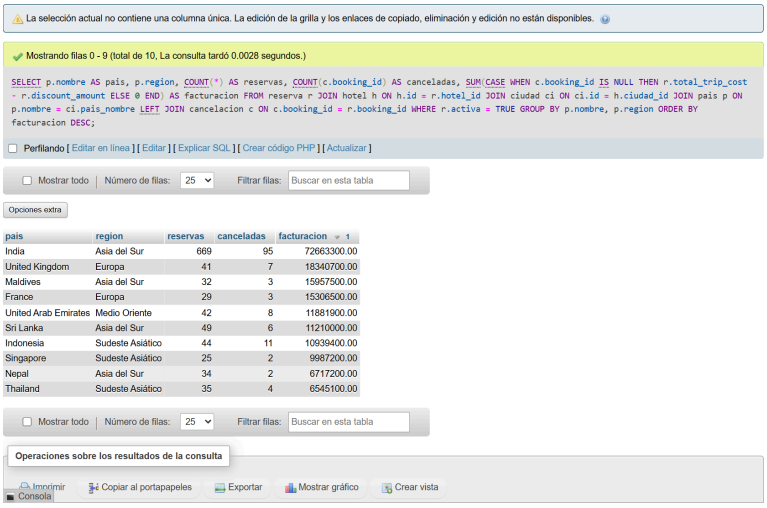


Figura 13. La consulta en phpMyAdmin. India: 669 reservas, 95 canceladas y 72.663.300 de facturación.

Consulta en MongoDB: promedio de las reseñas por alojamiento, con un aggregate sobre la colección resenas (los tres primeros por nombre):

```
db.resenas.aggregate([
  { $group: { _id: "$hotelId", hotel: { $first: "$hotel" }, resenas: { $sum: 1 },
      puntajeHotel: { $avg: "$puntajeHotel" }, puntajeViaje: { $avg: "$puntajeViaje" } } },
  { $project: { hotel: 1, resenas: 1, puntajeHotel: { $round: ["$puntajeHotel", 1] },
      puntajeViaje: { $round: ["$puntajeViaje", 1] } } },
  { $sort: { hotel: 1 } },
  { $limit: 3 }
])

{ _id: 1, hotel: 'Amber Heritage Stay', resenas: 14, puntajeHotel: 3.5, puntajeViaje: 3.2 }
{ _id: 2, hotel: 'Aravalli Palace Inn', resenas: 11, puntajeHotel: 2.9, puntajeViaje: 3 }
{ _id: 3, hotel: 'Azure Island Villas', resenas: 6, puntajeHotel: 3.6, puntajeViaje: 3.6 }
```

Consulta que usa las dos bases: la pantalla de estadísticas por alojamiento (Figura 14). Las reservas y las canceladas vienen de MySQL, y las reseñas y los puntajes de MongoDB. Amber Heritage Stay tiene 15 reservas, 1 cancelada y 14 reseñas con promedios de 3,5 y 3,2, los mismos números que da el aggregate.

Cómo le va a cada alojamiento

Los puntajes son el promedio de las reseñas que dejaron los huéspedes.

ALOJAMIENTO	RESERVAS	CANCELADAS	RESEÑAS	PUNTAJE DEL ALOJAMIENTO	PUNTAJE DE LA ESTADÍA
Amber Heritage Stay Jaipur, India	15	1	14	3,5	3,2
Aravalli Palace Inn Udaipur, India	13	2	11	2,9	3,0
Azure Island Villas Malé, Maldivas	8	1	6	3,6	3,6
Backwater Grand Kochi, India	9	4	5	3,5	3,2
Bali Lotus Villas Bali, Indonesia	16	4	11	2,8	2,8
Bangkok Garden Suites Bangkok, Tailandia	15	0	15	2,9	3,3
Beas Valley Resort Manali, India	12	1	10	2,5	3,3
Capital Grand Hotel Delhi, India	11	3	6	2,6	2,9

Figura 14. Estadísticas por alojamiento: datos de MySQL y de MongoDB en la misma tabla.

Por último, una reserva cargada desde la aplicación queda pendiente hasta que se confirma, y el total a pagar se calcula como precio menos descuento (Figura 15).

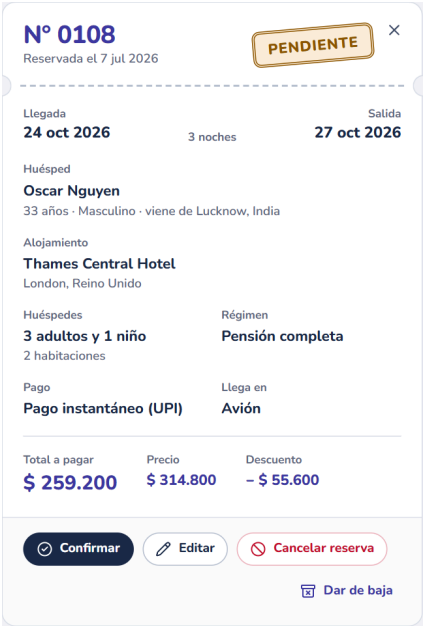


Figura 15. Una reserva pendiente, con el botón Confirmar y el total con el descuento aplicado.

14. Comparación entre ambas implementaciones

Aspecto	MySQL (reservas)	MongoDB (reseñas)
Modelo	11 tablas en 3FN	1 colección, un documento por reseña
Esquema	fijo, con CREATE TABLE	flexible, controlado con $jsonSchema
Integridad	PK, FK, UNIQUE y CHECK	validador, índice único y backend
Listas y datos opcionales	otra tabla o columnas en NULL	array y objeto opcional
Datos repetidos	ninguno	nombre y destino del hotel en cada reseña
Relaciones	JOIN entre tablas	no hay: el backend une por reservaId y hotelId
Consulta típica	JOIN y GROUP BY	aggregate con $group
Carga	INSERT ... SELECT desde la tabla del CSV	JSON armado con SQL y mongoimport

En la primera versión del modelo las reseñas también estaban en MySQL, en dos tablas: resena (puntajes, comentario y dos columnas para la respuesta, casi siempre en NULL) y resena_etiqueta (una fila por etiqueta). Para leer una reseña con sus etiquetas hacía falta un JOIN, y en MongoDB es un solo documento. A cambio, en MySQL la FK aseguraba que la reserva existiera; en MongoDB eso lo controla el backend. Con 1000 reservas las dos bases responden en milisegundos, así que la diferencia está en cómo se modelan los datos y no en la velocidad.

15. Problemas encontrados y decisiones tomadas

- Hotel_Rating cambia en cada reserva: es el puntaje del huésped, así que va en la reseña. Al principio el hotel guardaba una categoría base con el promedio, pero no se actualizaba con las reseñas nuevas; la sacamos y el promedio se calcula al consultar.

- El motivo de cancelación primero quedó como texto. Lo pasamos al catálogo motivo_cancelacion porque los 7 motivos se repiten, y así la aplicación solo deja elegir motivos de la lista.
- "Not Applicable" y puntajes en 0: los reemplazamos por la tabla opcional cancelacion y por las reseñas, que existen solo para las estadías completadas.
- La región no venía en el CSV: la agregamos con seis regiones.
- Primero hicimos un backend que guardaba todos los datos en las dos bases y elegía el motor con un selector. Lo cambiamos a un backend políglota: cada dato en la base que le conviene y las dos funcionando a la vez.
- La carga la hacía un script de Python que generaba el SQL. La pasamos a SQL (LOAD DATA INFILE e INSERT ... SELECT) para que la normalización se vea en el propio script. El JSON de las reseñas también sale de una consulta SQL.
- El CSV trae el costo y el descuento por separado. El total a pagar no se guarda porque se calcula, y la facturación de las estadísticas ya lo usa.
- Al principio el estado de la reserva se elegía en el formulario. Ahora toda reserva nueva empieza pendiente y se confirma con un botón.
- Los acentos se guardaban mal (Oceanía) porque el cliente de MySQL del contenedor usa latin1. Lo resolvimos con SET NAMES utf8mb4 al principio de la carga.
- La versión en la nube tarda en la primera carga porque el plan gratuito de Render apaga el servidor después de 15 minutos sin uso. No es por la cantidad de datos.
- La base de cada integrante en su Docker es una copia aparte: lo que carga uno no lo ve otro. Para probar juntos usamos la versión en la nube, que tiene una sola base.

16. Conclusiones

Normalizar sacó las repeticiones y las anomalías, y con PK, FK y CHECK la base misma rechaza datos inválidos. Por eso dejamos en MySQL todo lo que tiene relaciones y reglas fuertes: clientes, hoteles, reservas y cancelaciones. Las reseñas, con listas y datos opcionales, se adaptan mejor a MongoDB, aunque ahí la integridad depende del validador y del backend. Hacer la carga en SQL nos sirvió para ver la normalización paso a paso, y comparar los números de las dos bases nos sirvió para validar la carga. No hay un motor mejor en general: cada uno sirve para una parte del problema, y con una capa de persistencia con interfaces pueden convivir en la misma aplicación.